2025 International Ubiquitous Positioning, Indoor Navigation and Location-Based Services Conference (UPINLBS) | 979-8-3315-7138-2/25/$31.00 ©2025 IEEE | DOI: 10.1109/UPINLBS68186.2025.11468450

A Smartphone-Based Platform for Real-Time Deep Learning Pedestrian Dead Reckoning

Xuehang Sun[1], Yan Wang[2], You Li[1], Zhihao Liu[2]

[1] Wuhan University, State Key Laboratory of Information Engineering in Surveying, Mapping and Remote Sensing, Wuhan 430072, China – (xuehangsun, liyou)@whu.edu.cn
[2] Wuhan University, GNSS Research Center, Wuhan 430072, China – (wystephen, zhihao.liu)@whu.edu.cn

Keywords: Deep Learning, Smartphone Platform, Pedestrian Dead Reckoning, Indoor Positioning.

Abstract

Pedestrian Dead Reckoning (PDR) is crucial for indoor positioning when GNSS is not available. However, on-device deployment and unified testing platforms for deep learning–based PDR are still scarce. As a highly integrated and widely adopted terminal, the smartphone not only incorporates multimodal sensors such as IMU, camera, and GNSS, but also leverages a on-device computing resources capable of executing deep models locally. Thus, smartphones are a good choice for the development and testing of PDR. Based on this insight, this paper proposes a model conversion and lightweight deployment scheme and develops a deep learning PDR testing and visualization platform that can be installed on smartphones. The platform adopts a modular architecture that supports real-time execution of mainstream models and provides online visualization and logging of metrics such as global 2D trajectories, local 3D trajectories, and inference latency. Experiments show that on-device real-time inference is consistent with desktop offline inference in trajectory reconstruction and error evaluation. The proposed platform significantly reduces deployment and testing costs, promoting the practical implementation of deep learning–based PDR.

1. Introduction

Due to the absence of Global Navigation Satellite System (GNSS) signals in indoor environments, indoor positioning remains a challenging task. Existing indoor positioning techniques mainly include ultra-wideband positioning (UWB) based on distance trilateration, acoustic positioning (Lin et al., 2023), visible light positioning (Yang et al., 2025), radar positioning and camera-based positioning (Qin et al., 2018) using environmental characteristic matching, magnetic-field-based positioning, as well as wheel odometry (Wu et al., 2025) and inertial navigation (Kuang et al., 2025a) based on motion models. Among these methods, inertial-navigation-based approaches have attracted extensive attention because they do not require additional infrastructure deployment, do not rely on prior environmental knowledge, and can operate in various complex or even harsh environments. They are therefore often employed as a "bridge" in multi-sensor fusion systems.

For pedestrian indoor inertial navigation, systems can be roughly categorized according to the sensor installation location into foot-mounted (Kuang et al., 2025b), head-mounted(Liu et al., 2020), and hand-held configurations (Herath et al., 2020). From the perspective of positioning methodology, they can be further divided into data-driven methods, model-driven methods, and hybrid data–model-driven methods. In typical implementations, the inertial devices mounted on the feet and the head are customized hardware. In particular, for foot-mounted scenarios, pedestrian motion exhibits rich and regular characteristics, such as the approximate zero-velocity property of the foot during the stance phase and the fact that the toe orientation is usually consistent with the walking direction. By exploiting these motion characteristics, model-driven algorithms can provide additional observations or constraint information for inertial navigation, thereby enabling correction algorithms such as Zero-Velocity Update (ZUPT) and straight-line constraints, which effectively suppress error divergence.

In contrast, hand-held inertial devices impose more stringent requirements on the positioning algorithms. On the one hand, human hand motions are more diverse and complex. When the posture of the hand and the actions of the pedestrian are strictly constrained (for example, limited to a few specific patterns such as flat holding, natural arm swing, or calling), it is possible to construct the corresponding motion models in advance and achieve a certain level of positioning accuracy. On the other hand, once the pedestrian motion state goes beyond these predefined constraints, it becomes difficult for purely inertial positioning to maintain its accuracy. Under arbitrary pedestrian dynamics, the hand-held inertial signals are highly complex, with the lever arm and mounting angles changing frequently and irregularly. In such cases, traditional algorithms based on inertial mechanization, as well as methods relying on "step detection + step length estimation," almost cease to be effective.

In recent years, with the rapid development of deep learning, researchers have introduced deep learning techniques into the field of inertial navigation (Chen et al., 2018). By constructing deep neural networks that directly learn the mapping between inertial measurements and pedestrian velocity (or displacement), these methods can significantly improve the positioning accuracy and robustness of hand-held devices under arbitrary motion conditions, without the need to impose strict constraints on the motion state of the hand.

However, deep learning models typically have large numbers of parameters and incur high computational and energy costs, which places stringent requirements on the computational capability and power budget of terminal devices. As a result, most existing deep-learning-based PDR methods are evaluated in an offline post-processing manner after data collection, and there is a lack of a convenient, unified platform that supports on-demand real-time testing. Smartphones, as ubiquitous handheld devices, integrate multiple sensors including inertial sensors and thus provide a natural platform for the development and validation of positioning algorithms. Meanwhile, the rapid im-

Authorized licensed use limited to: Wuhan University. Downloaded on April 29,2026 at 13:30:54 UTC from IEEE Xplore. Restrictions apply.

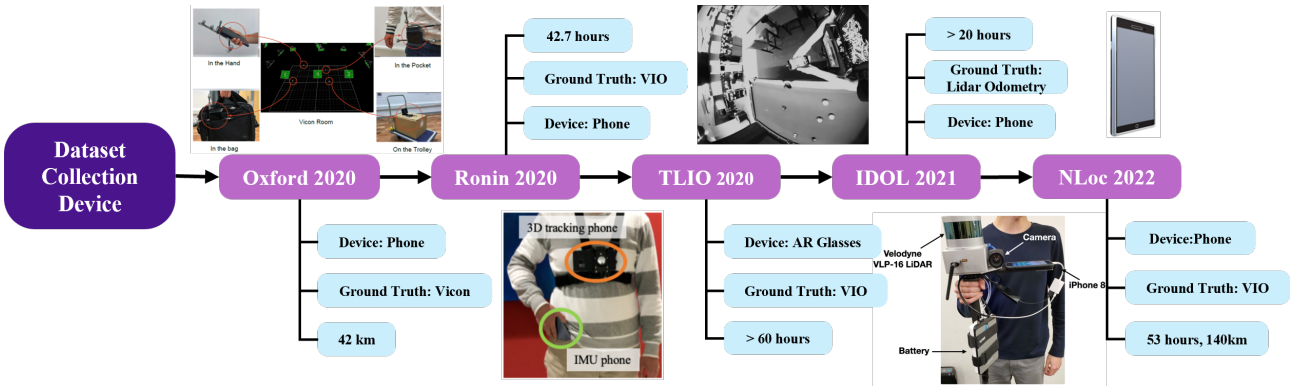


Figure 1. Data collection setups for deep-learning-based PDR datasets.

provement of smartphone CPU performance in recent years has made it possible to run real-time inference for most lightweight deep networks. Motivated by these observations, this paper designs and implements a smartphone-based platform to support real-time inference and evaluation of deep-learning-based PDR algorithms.

The remainder of this paper is organized as follows. Section 2 reviews related work on deep-learning-based PDR. Section 3 describes in detail the design of the proposed test platform. Section 4 presents experimental results obtained using this platform and analyzes the localization performance. Section 5 concludes the paper.

2. Related Work

Due to the high complexity of hand motion patterns and the fact that smartphones are carried by almost everyone, most existing deep-learning-based PDR studies use smartphones as the primary platform for inertial data collection and algorithm deployment. In the Oxford (Chen et al., 2020) dataset, optical motion capture reflective markers are attached to the smartphone, and an optical motion capture system is used to obtain high-precision ground truth, yielding smartphone-based pedestrian motion data with a total trajectory length exceeding 42 km. The RoNIN (Herath et al., 2020) dataset employs two smartphones, one mounted on the chest and the other handheld; the visual–inertial odometry (VIO) result of the chest-mounted phone is used as ground truth, and about 42.7 hours of data are collected. TLIO (Liu et al., 2020) uses a head-mounted AR headset as the inertial sensing device, again taking VIO as ground truth, and acquires more than 60 hours of inertial data without constraining pedestrian motion. IDOL (Sun et al., 2021) collects smartphone inertial data using a custom-built platform in which the phone is rigidly attached to a LiDAR sensor; LiDAR–inertial odometry serves as the ground truth, resulting in a high-precision dataset of more than 20 hours. NLoc (Herath et al., 2022) likewise uses a smartphone as the carrier for inertial data acquisition and adopts VIO as ground truth, collecting approximately 53 hours of data with a total distance over 140 km. The corresponding data collection setups are illustrated in Figure 1.

Although the above studies all use smartphones as the primary platform for deep learning–based PDR, their network designs mainly prioritize improving localization accuracy, while paying relatively limited attention to practical on-device deployment constraints such as computational capability and power budget. As a result, the models typically have large parameter sizes and high computational costs, making it difficult to run them continuously on battery-powered smartphones.

Table 1. Development and comparison of lightweight data-driven PDR models.

Model	Year	Device	Contributions
IONet	2018	Smartphone	Milestone
Ronin	2020	Smartphone	Milestone
L-IONet	2020	Smartphone	Wave-Net model
LLIO	2022	Smartphone	ResMLP model
SIMD	2023	Smartphone	TCN model
IMU-Net	2024	Smartphone	CNN model
ResMixer	2024	Smartphone	ResMLP model

To address the computational and energy bottlenecks of deploying data-driven PDR models on resource-constrained terminals, a number of recent works have explored lightweight solutions from the perspectives of both network architecture and input representation. L-IONet (Chen et al., 2020) is built on a WaveNet-style architecture and significantly reduces FLOPs by simplifying the network structure; LLIO (Wang et al., 2022) and IMUNet (Zeinali et al., 2024) adopt residual multilayer perceptrons (ResMLP) as the core backbone to improve parameter efficiency and maintain a compact model structure; SIMD (Liu et al., 2023) instead acts on the input side, manually designing features and reducing input dimensionality to directly alleviate the computational burden of the subsequent network; ResMixer (Lai et al., 2024) is likewise constructed on the ResMLP framework and further integrates UWB auxiliary information to enhance positioning performance. Among these lightweight approaches, LLIO achieves an approximately 12× speed-up in inference with almost no degradation in localization accuracy, and is one of the representative lightweight PDR models to date. The evolution and performance comparison of various data-driven lightweight PDR methods are summarized in Table 1.

Although existing deep-learning-based PDR studies on smartphone platforms are already quite rich, only a few works have actually deployed these models on smartphones to achieve real-time operation and demonstrate the resulting positioning performance. To more intuitively present the behavior of deep-learning-based PDR, we design and implement a real-time demonstration platform on smartphones, which has the following features:

1) it supports the deployment and real-time execution of multiple mainstream deep-learning PDR models;

2) it offers a simple deployment process and user-friendly operation;

Authorized licensed use limited to: Wuhan University. Downloaded on April 29,2026 at 13:30:54 UTC from IEEE Xplore. Restrictions apply.

3) it provides an intuitive visualization interface that can display local 3D position and attitude as well as the global 2D trajectory in real time.

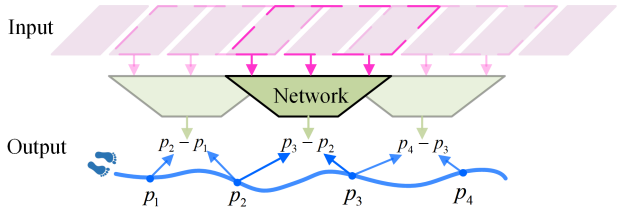


Figure 2. Network input–output configuration.

3. Methodoloy

This section presents the design of the test platform from three aspects: 1) the model generation scheme, 2) the platform design framework, and 3) implementation details. We first describe the input–output formats of the models to be deployed and their storage strategy, then elaborate on the overall architecture of the test platform and typical usage workflow, and finally provide key implementation details related to practical deployment and runtime operation.

3.1 Model Generation Scheme

In this work, all networks take IMU data sampled at 100 Hz as input, with a sliding window length of 1 s, and output the relative displacement within each window, as shown in the Figure 2.

This configuration is chosen for two reasons: first, a 1 s time window typically covers a nearly complete gait cycle and thus contains sufficiently rich pedestrian motion information; second, according to the Nyquist sampling theorem, an IMU sampling rate of 100 Hz can capture the main frequency components of normal pedestrian motion, thereby preserving the key temporal and spectral characteristics.

The relationship between the network inputs and outputs can be expressed as:

$$\hat{d}, \Sigma_{\hat{d}} = f(a^n_{\Delta t}, \omega^n_{\Delta t} | \theta) \tag{1}$$

where $a^n_{\Delta t}$ and $\omega^n_{\Delta t}$ denote the tri-axial acceleration and angular velocity in the navigation frame over the time interval Δt, respectively; $\boldsymbol{\theta}$ represents the learnable parameters of the network; and $\hat{\boldsymbol{d}}$ and $\boldsymbol{\Sigma}_{\hat{d}}$ denote the predicted 3D displacement and its associated covariance matrix, respectively.

To enable the network to jointly learn displacement and its uncertainty, we adopt a two-stage loss design during training: mean squared error (MSE) loss and negative log-likelihood (NLL) loss. In the first 20 epochs, only the MSE loss is used to promote fast convergence and obtain reasonable displacement estimates; thereafter, the network is trained with the NLL loss, where the Mahalanobis distance is employed to supervise the predicted covariance.

$$\mathscr{L}_{\text{MSE}}(d, \hat{d}) = \frac{1}{n} \sum \left\| d - \hat{d} \right\|^2 \tag{2}$$

where d denotes the displacement of ground truth, and $\hat{d}$ is the estimated displacement of the network.

$$\mathscr{L}_{\text{NLL}}(d, \hat{d}, \Sigma_{\hat{d}}) = \frac{1}{n} \sum \frac{1}{2} \log\left(\det\left(\Sigma_{\hat{d}}\right)\right) + \frac{1}{2} \left\| d_i - \hat{d} \right\|^2_{\Sigma_{\hat{d}}} \tag{3}$$

where $\boldsymbol{\Sigma}_{\hat{d}}$ denotes the displacement covariance. Furthermore, $\left\| d - \hat{d} \right\|^2_{\Sigma_{\hat{d}}}$ is defined as the squared Mahalanobis distance between $\boldsymbol{d}$ and $\hat{\boldsymbol{d}}$ under the covariance $\boldsymbol{\Sigma}_{\hat{d}}$. Using the NLL loss allows the network to jointly predict displacement and its covariance, providing confidence weights for the outputs and facilitating integration with filtering- or optimization-based methods. In contrast, traditional PDR methods do not produce covariance estimates and therefore cannot be directly applied in probabilistic fusion frameworks.

To enhance the model's generalization across different devices and environments, we employ the following data augmentation strategies during training:

- Apply a random horizontal rotation to the IMU measurements and the corresponding displacement simultaneously (within $[-180°, 180°]$), assuming that the AR headset can have an arbitrary heading with respect to the walking direction.
- Add a random bias to the IMU measurements (acceleration in $[-0.5, 0.5]$ m/s^2 and angular velocity in $[-0.5, 0.5]$ deg/s) to make the network robust to IMU bias.
- Introduce a random perturbation to the gravity direction (within $[0°, 5°]$) to increase robustness against errors in gravity orientation estimation.

After training the model, we use PyTorch Lightning to save both the network architecture and its parameters. Subsequently, for deployment purposes, the model is exported with TorchScript into a compact version that supports efficient inference on mobile devices and in C++ environments.

3.2 Platform Design Framework

The overall architecture and data flow of the test platform are illustrated in Figure 3, where the system is mainly composed of a front end and a back end.

The front end, implemented at the Java layer, is responsible for user interaction, real-time visualization of localization and model inference results, and invoking the smartphone APIs to start and relay IMU data. Considering the widespread use of C++ for algorithm development, the back end is implemented at the C++ layer, and the interaction between the front end (Java layer) and back end (C++ layer) is established via the Android NDK. The back end encapsulates the core localization pipeline, including data soft synchronization, algorithm initialization, model inference, and graph optimization; the optimization results are then returned to the front end through the NDK to display the user's real-time motion trajectory.

To facilitate model deployment and updates, the app, during initialization, first searches the phone's Download directory for a model file named cache.pt and, if found, loads it as the deep learning model. If the file is not present, the app falls back to an internal default model for initialization, which greatly simplifies the overall model deployment process.

Authorized licensed use limited to: Wuhan University. Downloaded on April 29,2026 at 13:30:54 UTC from IEEE Xplore. Restrictions apply.

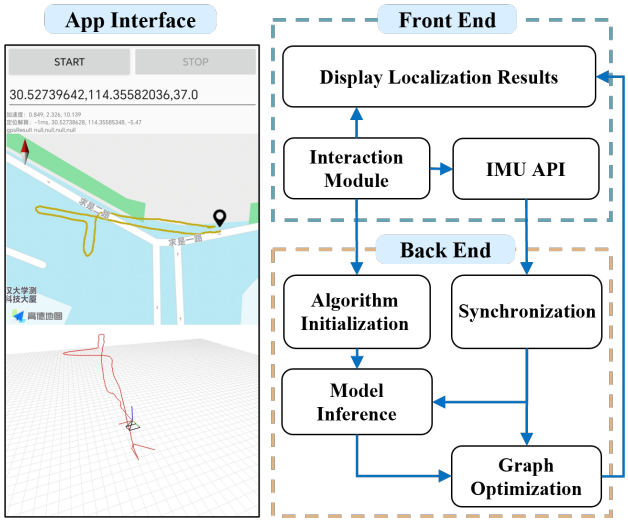


Figure 3. The platform's overall architecture and data flow.

The platform also integrates a rich set of third-party libraries, including but not limited to the libtorch deep learning library, the Ceres graph optimization library, the OpenCV computer vision library, and the Eigen linear algebra library, which together can support the deployment requirements of most mobile-side localization and perception algorithms.

3.3 Implementation Details

The proposed networks are implemented in PyTorch 1.8 and trained on an Ubuntu 20.04 workstation equipped with an Intel i5-13600KF CPU and an NVIDIA RTX 3090 GPU. All models evaluated in this work take 1 s IMU segments sampled at 100 Hz as input, with a sliding-window update rate of 10 Hz. During deployment, 2.1 s of continuous IMU measurements are collected and reorganized, according to the window size and update rate, into a tensor of shape [10,6,100], which is then fed into the CPU for a single forward inference pass. Therefore, in theory, as long as the forward-inference time for each batch is less than 1 s, the system can meet the real-time requirement of running a 1-second sliding window at a 10-Hz update rate.

For on-device deployment, we cross-compile the PyTorch source code with Android NDK 21.4.7075529, CMake 3.16.3, and Ninja 1.10.2 to generate static and shared libraries from libtorch targeting the `armeABIs-v7a` and `arm64-v8a` ABI, and use the same toolchain configuration to build the other native dependencies. To verify the applicability of the proposed test platform, we deploy and run it on multiple commercial smartphones, including Huawei Mate 40, Huawei Mate 40 Pro, Huawei Pura 70, Honor Magic 6, Honor 300, and Redmi K50 Ultra, all of which can execute the system stably.

4. Evaluation

We evaluate the proposed platform from three perspectives: 1) inference consistency, 2) inference latency, and 3) localization accuracy. Inference consistency verifies reliable cross-platform deployment, inference latency checks real-time capability, and localization accuracy evaluates performance against classical PDR methods.

Table 2. Inference latency (ms) for a 1000 ms window under different smartphone models and network configurations.

	ResMLP (7.5 MB)	IMUNet (12.7 MB)	ResNet (20.2 MB)
Huawei Mate40	227	952	917
Honor Magic6	247	965	902
Redmi K50 Ultra	349	-	-

4.1 Inference Consistency

Because the networks are trained and validated on the desktop using PyTorch, but deployed on smartphones via libtorch in C++, differences in hardware architecture, compilers, and numerical libraries may introduce subtle discrepancies in the outputs. If such discrepancies accumulate over multiple integration steps and graph optimization, they could lead to non-negligible deviations in the final trajectories. Therefore, before evaluating latency and localization accuracy, we first verify that the inference results on desktop and mobile platforms are numerically consistent under the same model and identical inputs.

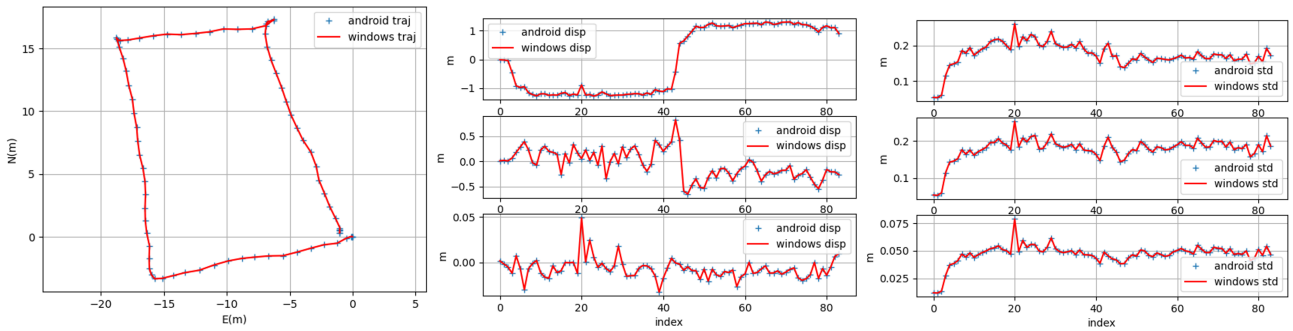
To verify the consistency between desktop and mobile inference, we run forward passes on the same trained network and identical IMU input sequences on both the desktop (PyTorch) and the Android smartphone (TorchScript/libtorch), and compare their outputs element-wise. Figures (a), (b), and (c) visualize the inference results from the two platforms: (a) shows the global 2D trajectories, comparing the overall walking paths; (b) presents the three-axis displacement time series, comparing the per-window displacement outputs; and (c) illustrates the three-axis covariance time series, comparing the estimated uncertainties. As can be observed, the curves from the two platforms almost perfectly overlap, and the element-wise numerical discrepancies are smaller than 6×10^{-7}. This demonstrates that, under the current deployment and compilation settings, the network inference on desktop and mobile devices is highly consistent in terms of numerical behavior, providing a reliable basis for subsequent latency and accuracy evaluations.

4.2 Inference Latency

Previous latency evaluations are often conducted under idealized conditions, where only the time of a single forward pass is measured with the smartphone CPU running at full speed, without accounting for the overhead of memory access, data preprocessing, and back-end optimization in the full pipeline. To obtain results that are more representative of real-world usage, after deploying all algorithmic modules on both the front end and back end, we measure the end-to-end runtime from IMU data input to trajectory output and evaluate the online inference latency of three popular deep learning models, ResMLP, IMUNet, and ResNet, on a unified platform across three smartphone configurations: Honor Magic 6, Huawei Mate 40, and Redmi K50 Ultra.

As shown in Table 2, all three deep learning networks can complete one forward pass within a 1 s time window on the tested smartphones, thereby meeting the real-time requirement. Among them, the lightweight ResMLP model achieves the lowest latency, about 227–247 ms on the Huawei Mate40 and Honor Magic6, and about 349 ms on the Redmi K50 Ultra, demonstrating good adaptability to different hardware configurations. In contrast, the per-window latency of IMUNet and ResNet on the first two phones is approximately 0.9 s to 1.0 s; although still within an

Authorized licensed use limited to: Wuhan University. Downloaded on April 29,2026 at 13:30:54 UTC from IEEE Xplore. Restrictions apply.



(a) (b) (c)

Figure 4. (a), (b), and (c) compare the inference results obtained on the Android smartphone and on the desktop under the same network model and identical input data. Subfigure (a) shows the global trajectory comparison, (b) presents the three-axis displacement inference results, and (c) illustrates the three-axis covariance inference results.

acceptable range, it is already close to the upper bound of real-time performance, indicating that a trade-off between network complexity and on-device computational constraints is still necessary.

4.3 Localization Accuracy

To highlight the advantages of deep-learning-based PDR and justify the necessity of the proposed platform, we compare three representative pedestrian localization methods: inertial-navigation mechanization, step-detection-based PDR, and deep-learning-based PDR, in order to evaluate the platform's localization accuracy in practical application scenarios. The experiments were conducted in an indoor area of approximately 20 m × 20 m, where the participant walked back and forth along multiple trajectories.

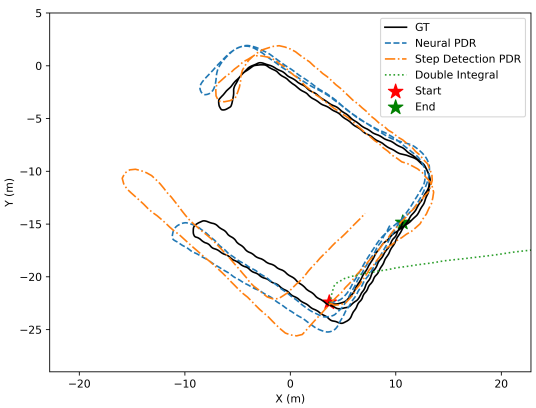


Figure 5. Comparison of inertial double integration, step-detection-based PDR, and deep-learning-based PDR.

Optimized English version: We evaluate the three methods on an approximately 3-minute indoor walking sequence that contains mixed short and long steps, introducing typical gait variations. A LiDAR odometry system is used as the ground-truth reference, and ResMLP (Wang et al., 2022) is adopted as the deep model in the Neural PDR method. Figure 5 compares the 2D trajectories of the three approaches, while Figure 6 presents the CDF curves of the positioning errors for the step-detection-based PDR and the Neural PDR. Among the three methods, inertial double integration diverges the fastest. The endpoint errors of the step-detection-based PDR and the Neural PDR are

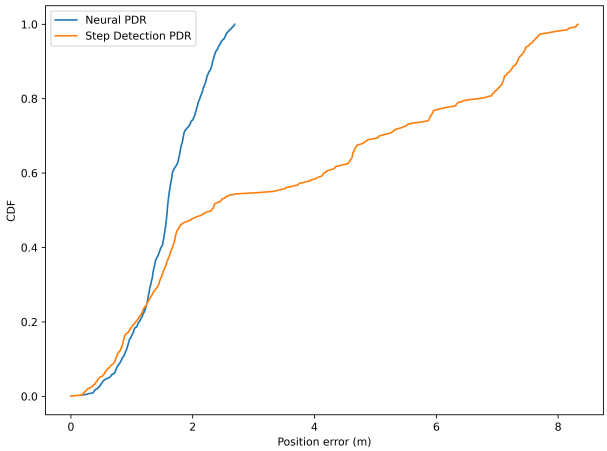


Figure 6. CDF of position errors.

4.3 m and 1.2 m, respectively, and their 95% CDF errors are 7.57 m and 2.46 m. After mixed step lengths appear, the step-detection-based PDR exhibits a clear scale mismatch, with noticeably over-extended steps in the lower-left part of the trajectory, whereas the Neural PDR maintains a more consistent trajectory scale and achieves better overall localization performance.

5. Conclusion

This paper addresses the need for visualization and evaluation of smartphone-based deep-learning PDR by proposing a complete model generation and on-device deployment scheme, and by implementing a real-time testing platform that runs on commercial smartphones. On the modeling side, the platform takes 1 s windows of 100 Hz IMU data as input, performs end-to-end joint learning of displacement and uncertainty with a two-stage loss (MSE + NLL), and incorporates data augmentation to improve generalization. On the deployment side, models trained on the desktop are migrated to Android devices via TorchScript and libtorch, and a user-friendly real-time demonstration system is built by combining a C++ back end with a Java front end.

Experimental results show that on-device inference is numerically highly consistent with desktop inference, with discrepancies on the order of less than 6×10^{-7}, and that real-time inference within a 1 s window can be achieved on multiple smart-

Authorized licensed use limited to: Wuhan University. Downloaded on April 29,2026 at 13:30:54 UTC from IEEE Xplore. Restrictions apply.

phones. Among the evaluated models, the lightweight ResMLP exhibits a clear latency advantage. Localization accuracy experiments further validate the practicality of the platform for pedestrian dead reckoning and provide a unified experimental environment for rapid iteration and fair comparison of future deep-learning-based PDR models.

In future work, we plan to incorporate additional sensing modalities, such as magnetometer and camera data, extend support to more network architectures and operating system versions, and systematically evaluate energy consumption characteristics. These efforts aim to further promote the deployment of deep-learning-based PDR in real indoor positioning and navigation scenarios.

References

Chen, C., Lu, X., Markham, A., Trigoni, N., 2018. Ionet: Learning to cure the curse of drift in inertial odometry. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32number 1.

Chen, C., Zhao, P., Lu, C. X., Wang, W., Markham, A., Trigoni, N., 2020. Deep-learning-based pedestrian inertial navigation: Methods, data set, and on-device inference. *IEEE Internet of Things Journal*, 7(5), 4431–4441.

Herath, S., Caruso, D., Liu, C., Chen, Y., Furukawa, Y., 2022. Neural inertial localization. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 6604–6613.

Herath, S., Yan, H., Furukawa, Y., 2020. Ronin: Robust neural inertial navigation in the wild: Benchmark, evaluations, & new methods. *2020 IEEE international conference on robotics and automation (ICRA)*, IEEE, 3146–3152.

Kuang, J., Ding, L., Wang, Y., Yuan, Y., Jiang, P., Zhou, Q., Niu, X., 2025a. A Robust INS State Initialization Method for Vehicular GNSS/MEMS-INS Integrated Navigation in Urban Environment. *IEEE Transactions on Intelligent Transportation Systems*.

Kuang, J., Liu, T., Wang, Y., Meng, X., Niu, X., 2025b. Magnetic vector constraint pedestrian dead reckoning based on foot-mounted and waist-mounted imu. *IEEE Internet of Things Journal*.

Lai, R., Tian, Y., Tian, J., Wang, J., Li, N., Jiang, Y., 2024. ResMixer: A lightweight residual mixer deep inertial odometry for indoor positioning. *IEEE Sensors Journal*.

Lin, X., Chen, R., Huang, L., Liu, Z., Niu, X., Guo, G., Li, Z., Qian, L., 2023. ChirpTracker: A precise-location-aware system for acoustic tag using single smartphone. *IEEE Internet of Things Journal*, 11(1), 848–862.

Liu, F., Ge, H., Tao, D., Gao, R., Zhang, Z., 2023. Smartphone-based pedestrian inertial tracking: Dataset, model, and deployment. *IEEE Transactions on Instrumentation and Measurement*, 73, 1–13.

Liu, W., Caruso, D., Ilg, E., Dong, J., Mourikis, A. I., Daniilidis, K., Kumar, V., Engel, J., 2020. Tlio: Tight learned inertial odometry. *IEEE Robotics and Automation Letters*, 5(4), 5653–5660.

Qin, T., Li, P., Shen, S., 2018. Vins-mono: A robust and versatile monocular visual-inertial state estimator. *IEEE transactions on robotics*, 34(4), 1004–1020.

Sun, S., Melamed, D., Kitani, K., 2021. Idol: Inertial deep orientation-estimation and localization. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35number 7, 6128–6137.

Wang, Y., Kuang, J., Niu, X., Liu, J., 2022. LLIO: Lightweight learned inertial odometer. *IEEE Internet of Things Journal*, 10(3), 2508–2518.

Wu, Y., Kuang, J., Niu, X., Stachniss, C., Klingbeil, L., Kuhlmann, H., 2025. Wheel-GINS: A GNSS/INS Integrated Navigation System with a Wheel-mounted IMU. *IEEE Transactions on Intelligent Transportation Systems*.

Yang, X., Zhang, H., Zhuang, Y., Wang, Y., Shi, M., Xu, Y., 2025. ulidr: An inertial-assisted unmodulated visible light positioning system for smartphone-based pedestrian navigation. *Information Fusion*, 113, 102579.

Zeinali, B., Zanddizari, H., Chang, M. J., 2024. Imunet: Efficient regression architecture for inertial imu navigation and positioning. *IEEE Transactions on Instrumentation and Measurement*.

Authorized licensed use limited to: Wuhan University. Downloaded on April 29,2026 at 13:30:54 UTC from IEEE Xplore. Restrictions apply.